P.Vishwa nandan(192473015)

Q1 – Building Text Under Load: String, StringBuilder or StringBuffer?

Marks: 6

1. Introduction

A railway reservation system generates e-tickets containing information such as:

- PNR number
- Passenger name
- Coach number
- Berth number
- Fare
- Journey details

These details must be combined into a single printable text.

Java provides three ways to build strings:

1. String
2. StringBuilder
3. StringBuffer

2. Prediction Before Implementation

A. String

String is immutable, which means its value cannot be changed after creation.

When we concatenate strings using +, Java creates new String objects.

Repeated concatenation inside a loop creates many intermediate objects and requires additional memory allocation.

B. StringBuilder

StringBuilder is mutable, which means its content can be modified without creating a new String object for every append operation.

It provides faster string construction for repeated modifications.

C. StringBuffer

StringBuffer is similar to StringBuilder, but its methods are synchronized.

It is thread-safe for individual operations, but synchronization may introduce additional overhead.

Predicted Behaviour

Feature	String	StringBuilder	StringBuffer
Mutability	Immutable	Mutable	Mutable
Speed	Slower for repeated concatenation	Generally faster	Slightly slower than StringBuilder
Thread safety	Immutable	Not synchronized	Synchronized methods
Memory usage	More intermediate objects	Fewer intermediate objects	Fewer intermediate objects
Best use	Fixed text	Repeated text building	Shared mutable text

3. Design Three Workloads

We will use three different workloads to compare performance.

Workload 1 – Small Ticket

- Number of tickets: 1
- Fields per ticket: 6
- Purpose: Compare performance for a small ticket.

Workload 2 – Large Ticket Generation

- Number of tickets: 10,000
- Fields per ticket: 50
- Purpose: Compare performance under heavy workload.

Workload 3 – Many Short Tickets

- Number of tickets: 50,000
- Fields per ticket: 10
- Purpose: Test performance when many small tickets are generated.

These workloads help identify how each approach behaves as the number of fields and tickets increases.

Important note about measurements

The program measures average execution time across five runs after warm-up.

The memory measurement estimates the retained heap used by the completed ticket strings. It does not measure all temporary objects or peak memory during execution. Garbage collection can also affect the result.

For the intermediate String objects, use the theoretical estimate discussed in the next section.

5. Expected Behaviour and Experimental Comparison

Run the program and record the actual results from your computer.

Use this table in your assignment:

Workload	String time	StringBuilder time	StringBuffer time
1 ticket, 6 fields	Record output	Record output	Record output
10,000 tickets, 50 fields	Record output	Record output	Record output
50,000 tickets, 10 fields	Record output	Record output	Record output

Predicted observations:

1. For small tickets, the performance difference may be negligible.
2. For large workloads, StringBuilder is expected to perform better than repeated String concatenation.
3. StringBuffer may be slightly slower than StringBuilder due to synchronization.
4. Repeated String concatenation creates more intermediate objects.

Do not fill this table with estimated numbers. Use the actual measurements printed by your program.

6. String Immutability and Intermediate Object Creation

Consider:

String ticket = "";

ticket = ticket + "PNR";

ticket = ticket + "Passenger";

ticket = ticket + "Fare";

Each concatenation creates a new String result.

For n fields, repeated concatenation creates approximately n intermediate result strings, although exact allocation counts depend on the Java compiler and runtime.

With StringBuilder:

```
StringBuilder ticket = new StringBuilder();


ticket.append("PNR");
ticket.append("Passenger");
ticket.append("Fare");


String result = ticket.toString();
```

The same mutable buffer is reused, avoiding a new result String for every append.

Complexity Analysis

Let n represent the number of fields and assume field lengths are approximately equal.

String using repeated +:

- Time complexity: $O(n^2)$ character copying.
- Space complexity: O(n) temporary intermediate String objects in the simple estimate.
- More memory allocation and garbage collection.

StringBuilder:

- Time complexity: O(n) in the number of total output characters.
- Space complexity: O(n) for the growing buffer and final result.
- Fewer intermediate objects.

StringBuffer:

- Time complexity: O(n) in total output characters, with synchronization overhead.
- Space complexity: O(n).
- Thread-safe individual operations.

More precisely, if the final ticket contains L characters, StringBuilder and StringBuffer require O(L) construction time and storage. Repeated concatenation can require $O(L^2)$ character copying when appending similarly sized fields.

7. Design Decision for a Multithreaded Booking Server

The railway system uses multiple threads to generate tickets and update a shared booking log.

For individual e-ticket generation: StringBuilder

StringBuilder is recommended because each booking thread can create its own local StringBuilder object.

It provides efficient text construction without requiring synchronization because the builder is not shared between threads.

For the shared booking log: StringBuffer

StringBuffer is suitable because its methods are synchronized and protect individual append operations from concurrent modification.

However, if an entire multi-part log record must remain together, the complete append sequence should be synchronized.

A production logging framework or a dedicated logging queue would also be a suitable alternative.

Recommendation

Requirement	Recommended class	Reason
Individual ticket	StringBuilder	Fast and efficient
Shared mutable booking log	StringBuffer	Synchronized operations
Fixed text	String	Immutable and simple

8. Alternative Concept – String.join(), StringJoiner and String.format()

Instead of StringBuilder, Java provides other methods to combine text.

A. String.join()

```
String ticket = String.join(" | ",
        "PNR123",
        "Arun",
        "A1",
        "Berth 25",
        "Rs. 1500");
```

It joins multiple strings using a specified delimiter.

B. StringJoiner

```
import java.util.StringJoiner;


StringJoiner ticket = new StringJoiner(" | ");
```

```
ticket.add("PNR123");

ticket.add("Arun");

ticket.add("A1");

ticket.add("Berth 25");

ticket.add("Rs. 1500");


System.out.println(ticket);
```

StringJoiner is useful when combining multiple fields with a delimiter.

C. String.format()

```
String ticket = String.format(
        "PNR: %s | Passenger: %s | Fare: Rs. %.2f",
        "PNR123",
        "Arun",
        1500.0);
```

String.format() is useful when the output needs a fixed formatting pattern.

Complexity Comparison

Method	**Time complexity**	**Main advantage**
Repeated String +	$O(n^2)$ character copying	Simple syntax
StringBuilder	O(L)	Efficient repeated appending
StringBuffer	O(L)	Synchronized operations
String.join()	O(L) expected	Simple joining
StringJoiner	O(L) expected	Delimiter-based joining
String.format()	Generally O(L) output work	Formatted output

Here, L represents the total number of output characters.

The exact execution time depends on implementation and input, so actual benchmarking is required.

9. Conclusion

String, StringBuilder and StringBuffer serve different purposes in Java. String is suitable for fixed text, while StringBuilder is preferred for repeated string modifications. StringBuffer is useful when multiple threads share a mutable string. Experimental testing helps identify performance differences under different workloads. For the railway reservation system, StringBuilder is recommended for individual ticket generation and StringBuffer for a shared mutable booking log.